Composite

Season: 2 · Structural · Category: GoF · Level: Beginner

Intent

- Treat individual objects and compositions uniformly.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Folders contain files and folders.

When to Use

- See Intent.
- The design force named in Intent shows up more than once.

When Not to Use

- When a simpler redesign removes the need.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.gof.structural.composite;

import java.util.ArrayList;
import java.util.List;

sealed interface FileSystemNode permits FileLeaf, DirectoryComposite {
    String name();
    int size();
}

record FileLeaf(String name, int size) implements FileSystemNode {}

public final class DirectoryComposite implements FileSystemNode {
    private final String name;
    private final List<FileSystemNode> children = new ArrayList<>();

    public DirectoryComposite(String name) { this.name = name; }

    public DirectoryComposite add(FileSystemNode node) {
        children.add(node);
        return this;
    }

    @Override public String name() { return name; }

    @Override public int size() {
        return children.stream().mapToInt(FileSystemNode::size).sum();
    }
```

Classic Implementation (2)

```
}
```

Modern Java 25 / Functional

- Prefer sealed interfaces + records; virtual proxy uses synchronized lazy init.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: gof/structural/composite/
```

Pitfalls

- Keep adapters thin.
- Decorators should preserve the interface.
- Facades must not become god objects.

Interview Q&A

- Q: When is Composite the wrong choice? A: When you can change the adaptee/subject API or collapse the hierarchy.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Treat individual objects and compositions uniformly.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Folders contain files and folders. [0:40] Intent — State the design force in one sentence: Treat ind...